Assignment 2 (CSI-4107)

Members of Group 25

Member	Student Number
Nalan Kurnaz	300245521
Alona Petrova	300074852
Kevin Luong	300232125

Contributions

Member	Contributions
Nalan Kurnaz	- Research on the IR systems - Report writing
Alona Petrova	- Revision and local testing - Report writing
Kevin Luong	- Implementation of the pipeline - Setting up 11 experiements - Evaluation of the results - Work on report

Introduction

In this project, we aim to develop an improved version of the Information Retrieval (IR) system implemented in Assignment 1 by integrating recent neural information retrieval techniques. The primary objective is to achieve better evaluation scores through the use of deep learning models, including transformers, BERT-like architectures, and Large Language Models (LLMs).

Recap of Assignment 1

In Assignment 1, we implemented a traditional IR system using the **BM25+** ranking function. The system was evaluated by calculating the **Mean Average Precision (MAP)** for two types of queries:

- **Titles-only Queries:** Achieved a MAP score of **0.2938**.
- **Titles and Full-text Queries:** Achieved a significantly higher MAP score of **0.5485**.

The results demonstrated that including full-text indexing improved retrieval performance due to the availability of richer contextual information. Titles alone provided limited context, resulting in fewer relevant term matches, while full-text indexing allowed for greater term overlap and semantic variation, leading to better ranking of relevant documents.

Objective of Assignment 2

The goal of this assignment is to leverage recent advances in neural IR models to enhance retrieval performance beyond the scores obtained in Assignment 1. By incorporating deep learning approaches such as transformers and BERT-like models, we aim to improve query understanding, semantic matching, and overall retrieval quality.

Functionality Overview

1. Haystack Framework

Haystack is an open-source NLP framework designed for building scalable and production-ready information retrieval (IR) pipelines. It allows for the construction of modular pipelines that integrate different components such as retrievers, rankers, generators, and embedders to create a complete IR system. Haystack supports various document stores (such as Elasticsearch, FAISS, and InMemory stores), retrievers (BM25, embedding-based, etc.), and re-rankers (transformer models).

The following documentation was helpful during implementation of the pipeline:
https://haystack.deepset.ai/cookbook/query-expansion

How We Leveraged Haystack:

- **Document Storage:** Used `InMemoryDocumentStore` to store BM25 and embedding-based document representations.
- **Preprocessing:** Applied `DocumentCleaner` and `DocumentSplitter` to clean and split the text into manageable chunks.
- **BM25 Retrieval:** Utilized `InMemoryBM25Retriever` for fast retrieval using BM25+.
- **Embedding Retrieval:** Used `SentenceTransformersDocumentEmbedder` and `InMemoryEmbeddingRetriever` for semantic search.
- **LLM Integration:** Integrated Google's Gemini API with `GoogleAIGeminiGenerator` for query expansion.
- **Ranking and Re-ranking:** Applied `TransformersSimilarityRanker` for cross-encoder-based document re-ranking after initial retrieval.

2. Program Overview

The system follows a modular, pipeline-based architecture with the following components:

Preprocessing Pipeline

- **Document Cleaning:**
  - Removes unnecessary whitespaces and repeated substrings.
  - Normalizes Unicode text to ensure consistency.
- **BM25 Formatting:**
  - Formats the text to be compatible with BM25 scoring by ensuring the appropriate structure for term frequency-based scoring.
- **Sentence Embedding:**
  - Splits documents into chunks (3 sentences with overlap).
  - Embeds the text using `sentence-transformers/all-MiniLM-L12-v2` to generate vector embeddings.

BM25 Pipeline

- **Query Expansion:**
  - Expands user queries with relevant terms using the Gemini API.
- **BM25 Retrieval:**
  - Retrieves the top `k` documents using BM25+ with improved term-frequency-based scoring.
- **Scaling Scores:**
  - Normalizes BM25 scores to enable combination with embedding-based scores effectively.

Embedding Pipeline

- **Text Embedding:**
  - Converts query text into vector embeddings.
- **Cosine Similarity Retrieval:**
  - Retrieves top similar documents by comparing query embeddings with stored document embeddings.

Ranking Pipeline

- **BM25 and Embedding Score Combination:**
  - Combines BM25 and embedding scores using a weighted sum where `bm25_weight` and `embedding_weight` control the relative importance of each.
- **Document Splitting:**
  - Splits documents into smaller chunks to improve ranking accuracy.
- **Cross-Encoder Re-ranking:**
  - Uses a `TransformersSimilarityRanker` to perform final re-ranking by computing semantic similarity between query and document pairs.

Result Generation

- **Duplicate Removal:**
  - The `drop_duplicates` function is used to ensure that documents with multiple similar chunks are de-duplicated, retaining the highest-ranked version based on the document score. This step ensures that only the best version of each document is kept in the final results.
- **Result Storage:**
  - Outputs results to in TREC format.

Get Started / Instructions on How to Run the System

Prerequisites

Before running the system, make sure you have the following:

1. **Python**: Ensure you have Python 3.7 or later installed on your local machine.

2. **Gemini API Key**: The system relies on the Gemini model from Google AI Studio. You need an API key to access the Gemini model. Follow these steps:
    - Go to Google AI Studio to retrieve the API key
3. **TREC Eval Tool**: This tool is used to evaluate the results of the IR system. Follow these steps to set it up:
    - Download the trec_eval tool from the official repository.
    - Extract the `.tar` file using a tool such as `tar` (Linux/macOS) or `7-Zip` (Windows).
    - Ensure both `trec_eval` and your code are in the same directory for easier execution.
    - Compile the `trec_eval` tool:
        - On POSIX systems, use:

```
cd trec_eval-9.0.7
make
```

        - On MinGW/GCC, use:

```
gcc -o trec_eval trec_eval.c
```

4. **Required Python Libraries**: Ensure you have all the necessary dependencies by installing them via `pip`. Run the following command to install the dependencies:

```
pip install -r requirements.txt
```

Please note that some packages can conflict hence it is recomended to create a separate environment for testing the code.

Running the System

1. **Run the Python Script**:

- Once all dependencies are installed and the Gemini API is set up, modify the name of the output document destination and the name, and run the program using:

```
python haystack_pipeline.py
```

2. **Evaluating the Results**:

- After running the system, the results will be saved in a file (e.g., `results_triad_v3.txt`).
- To evaluate the results, you need to use the `trec_eval` tool. First, copy the `scifact/qrels/test.txt` and the BM25 result file (e.g., `bm25_result_for_titles.txt`) to the same directory where `trec_eval` is located.

- Run the following command to evaluate the results:

```
./trec_eval test.txt <bm25_result_file>
```

Replace `<result_file>` with the name of your result file, for example:

```
./trec_eval test.txt bm25_result_for_titles.txt
```

Notes:

- **Google AI Studio API**: Ensure that your API key is correctly set in your env variables. The key is used by the system to access the Gemini model for query expansion.
- The program sleeps for 60 seconds after every 30 queries to avoid hitting Google AI Studio's rate limits.

Algorithms, Data Structures, and Optimization

1. Algorithms

1.1 **Document Preprocessing**

The preprocessing pipeline involves cleaning and formatting documents to prepare them for downstream tasks. The key operations in this pipeline are:

- **Document Cleaning**: The document content is processed to remove unnecessary whitespaces, repeated substrings, and empty lines. This is achieved using the `remove_empty_lines`, `remove_extra_whitespaces`, and `remove_repeated_substrings` parameters in the `DocumentCleaner` component. The cleaning process ensures that the text is uniform and ready for subsequent processing.
- **Unicode Normalization**: Ensures consistency in Unicode text by using the `unicode_normalization="NFKC"` parameter in the DocumentCleaner. This guarantees that characters with different Unicode representations but the same visual appearance are standardized.
- **Text Chunking**: The document is split into smaller chunks (sentences) for efficient processing, especially for embedding models and retrieval mechanisms. This is done using the `DocumentSplitter` with parameters like `split_by="sentence"`, `split_length=3`, and `split_overlap=2`. This approach helps in breaking down long documents into smaller parts that can be processed more easily by embedding models.

1.2 **BM25 Retrieval Algorithm**

The BM25 pipeline focuses on retrieving relevant documents based on term frequency. It consists of the following components:

- Query Expansion: User queries are expanded using the `QueryExpander` component, which leverages the Google AI Gemini API to expand the query with relevant terms, increasing the recall for the retrieval step.

- BM25 Retrieval: The `InMemoryBM25Retriever` is used for BM25-based document retrieval. It relies on term frequency and inverse document frequency (TF-IDF) scoring to retrieve the most relevant documents. The parameters for the retrieval can be controlled with `top_k` to limit the number of documents returned.
- Scaling Scores: The BM25 scores are normalized to combine effectively with embedding-based scores. This scaling helps in achieving a balance between the two retrieval approaches (BM25 and embedding retrieval).

1.3 **Embedding Pipeline**

The embedding pipeline uses pre-trained models to convert text into vector representations, enabling semantic search and document retrieval based on meaning rather than exact term matches.

- Text Embedding: The SentenceTransformersDocumentEmbedder is used to convert document content into vector embeddings. The model `sentence-transformers/all-MiniLM-L12-v2` is utilized for embedding generation.
- Cosine Similarity Retrieval: The `InMemoryEmbeddingRetriever` retrieves documents based on cosine similarity, comparing the query's embedding with the stored document embeddings to find the most similar documents.

1.4 **Ranking Pipeline**

The ranking pipeline combines the BM25 and embedding scores to generate a final ranking for the documents. This is achieved by the following steps:

- **BM25 and Embedding Score Combination**: The `BM25AndEmbedderRanker` combines BM25 and embedding-based retrieval scores by assigning weights (`bm25_weight` and `embedding_weight`). The relative importance of each component can be adjusted based on the desired performance.
- **Document Splitting**: The `DocumentSplitter` is used once again to split the documents into smaller chunks to improve ranking accuracy
- **Cross-Encoder Re-ranking**: The `TransformersSimilarityRanker` uses a pre-trained model to re-rank the documents based on the semantic similarity between the query and the documents. This helps refine the ranking by considering both BM25 and embedding-based features.

2. **Data Structures**

The system uses various data structures to manage and process documents efficiently throughout different stages of the pipeline. The primary data structures include:

2.1 **Document Object**

- **Structure:** Each document is represented as a dictionary-like object containing fields such as:
    - `content` – The actual text of the document.
    - `meta` – Metadata associated with the document, including fields like `name`, `source`, or any custom attributes.
    - `embedding` – Vector representation of the document used in embedding retrieval.
- **Usage:**

- During preprocessing, documents are cleaned and chunked using `DocumentSplitter`.
- BM25 and embedding models use these documents for retrieval and ranking.

2.2 **Document Store**

- **Structure:** The document store holds processed documents for retrieval. Two types of document stores are used:
  - `InMemoryDocumentStore` – Stores documents in memory and supports fast retrieval.
  - `BM25DocumentStore` – Stores documents in a format optimized for BM25 retrieval.
- **Usage:**
  - Preprocessed documents are stored using `document_store.write_documents()` to make them accessible for subsequent retrieval.

2.3 **Embedding Index**

- **Structure:** The embedding index stores vector representations of documents generated by the embedding model. It maintains a mapping between document IDs and their corresponding embeddings for efficient similarity search.
- **Usage:**
  - The `InMemoryEmbeddingRetriever` performs cosine similarity comparisons using this index to retrieve relevant documents.

2.4 **Query Object**

- **Structure:** A query object holds the user's input query with optional expanded terms for improved retrieval. It may include:
  - `query` – Original user query.
  - `expanded_query` – Expanded query terms generated by the `QueryExpander`.
- **Usage:**
  - The query object is passed through BM25 and embedding retrievers for relevant document retrieval.

2.5 **Ranked Document List**

- **Structure:** The ranked document list stores retrieved documents, each with associated relevance scores. Each document object includes:
  - `document` – The original document object.
  - `score` – The relevance score assigned by the retriever or ranker.
- **Usage:**
  - This list is used in the ranking phase, where scores from different retrievers (BM25 and embeddings) are combined.

2.6 **Chunked Document List**

- **Structure:** When documents are split using `DocumentSplitter`, the resulting chunks are stored as a list where each chunk is treated as an individual document.
- **Usage:**
    - These chunks are passed through the embedding retriever and ranker to improve the quality of search results.

2.7 **Re-ranker Object**

- **Structure:** Stores the document-query pairs that need to be re-ranked by the cross-encoder model.
- **Usage:**
    - This object is passed to the `TransformersSimilarityRanker` to refine the document order based on semantic similarity.

3. **Optimization Techniques**

Several optimization techniques are implemented in the system to enhance efficiency, reduce latency, and improve retrieval accuracy.

3.1 **Document Chunking**

- **Purpose:** Improves retrieval efficiency by splitting large documents into smaller, manageable chunks.
- **Implementation:**
    - `DocumentSplitter` splits documents into sentences or paragraphs based on a configurable chunk size.
    - Smaller chunks allow for more precise retrieval, reducing noise and improving embedding relevance.
- **Optimization Impact:**
    - Decreases search space and ensures embeddings are computed on smaller, meaningful text units.

3.2 **BM25 Pre-filtering**

- **Purpose:** Reduces the number of documents passed to the embedding retriever by filtering top-ranked documents using BM25.
- **Implementation:**
    - BM25 returns a subset of top `k` documents with the highest relevance scores.
    - Only these documents are passed to the `InMemoryEmbeddingRetriever` for further refinement.
- **Optimization Impact:**
    - Reduces the computational load of embedding comparisons and re-ranking.

3.3 **Hybrid Retrieval Pipeline**

- **Purpose:** Combines the strengths of BM25 and embedding retrieval to balance precision and recall.
- **Implementation:**

    - BM25 quickly retrieves relevant keyword-based documents.
    - Embedding retrieval handles semantic matching and similarity comparison.
    - Results from both retrievers are merged and re-ranked for optimal relevance.
- **Optimization Impact:**
    - Improves overall retrieval performance by leveraging complementary strengths of BM25 and embeddings.

3.4 **Embedding Caching**

- **Purpose:** Avoids redundant embedding computations for previously processed documents.
- **Implementation:**
    - Embeddings are generated once and stored in the `embedding_index`.
    - Subsequent queries use precomputed embeddings, reducing processing time.
- **Optimization Impact:**
    - Significantly reduces embedding generation overhead for frequently accessed documents.

3.5 **Parallel Processing**

- **Purpose:** Reduces preprocessing and retrieval time by utilizing multi-threading or parallelism.
- **Implementation:**
    - Parallel processing is applied in document chunking, embedding computation, and BM25 retrieval.
- **Optimization Impact:**
    - Improves system throughput and minimizes latency during high-volume document processing.

3.6 **Re-ranker Thresholding**

- **Purpose:** Prevents re-ranking unnecessary documents by limiting the number of candidate documents passed to the re-ranker.
- **Implementation:**
    - A threshold `top_n` is applied to filter high-scoring documents before re-ranking.
- **Optimization Impact:**
    - Reduces the computational cost of cross-encoder re-ranking by focusing only on the most relevant candidates.

3.7 **Efficient Memory Management**

- **Purpose:** Minimizes memory consumption by optimizing data storage and retrieval mechanisms.
- **Implementation:**
    - `InMemoryDocumentStore` uses lightweight storage techniques.
    - Large intermediate results are discarded after use to avoid memory bloat.
- **Optimization Impact:**
    - Ensures scalability and efficiency when handling large document collections.

Results Overview and Discussion

The evaluation of different Information Retrieval (IR) systems was conducted using standard metrics: **Mean Average Precision (MAP)** and **Precision at 10 (P@10)**. Below is a summary of the results, along with a brief

description of each approach:

IR System	MAP	P@10	Description
BM25 (assignment 1)	0.5485	0.0847	Standard BM25 model using default parameters for relevance scoring.
BM25_Haystack	0.4995	0.0750	BM25 implementation using Haystack, with minimal parameter adjustments.
Embedding_Haystack	0.6031	0.0883	Dense embedding retrieval using Haystack's embedding retriever.
Hybrid Ver2	0.5949	0.0873	Combines BM25 and sentence transformers (`all-MiniLM-L6-v2`) with a weighted sum (0.3 BM25, 0.7 embeddings).
Hybrid_Ver2_Expansion	0.6095	0.0877	Same as Hybrid_Ver2 but includes query expansion using the Gemini model before applying BM25.
Triad	0.6379	0.0923	Extends Hybrid_Ver2 by adding a semantic text ranker (`cross-encoder/ms-marco-MiniLM-L-6-v2`)
Triad_v2	0.6519	0.0913	Fixes bugs in Triad by using original (unprocessed) documents for ranking and switches to larger models for embeddings (`all-MiniLM-L12-v2`) and ranking (`all-MiniLM-L12-v2`).
Triad_v3	**0.6605**	**0.0943**	Builds on Triad_v2 by splitting documents into chunks of 3 sentences with 2-sentence overlap to prevent truncation and removes duplicate documents after ranking **The best performing system.**
Triad_v4	0.6583	0.0947	Changes the ranking model to a larger model (`cross-encoder/ms-marco-electra-base`) and reduces document splitting to 2 sentences with 1-sentence overlap.
Triad_v5	0.6515	0.0947	Reverts to Triad_v3 but reduces document splitting to 2 sentences with 1-sentence overlap.

The result for the best performing model was saved under the name `results_triad_v3.txt`

In this report, we have structured the results and discussion into three sections:

1. **Comparison of the best-performing IR system with the Assignment 1 IR system.**
2. **Comparison of different approaches and neural models used to achieve the best performance in this assignment.** Specifically, we compare Triad_v3 (the best performer) with Triad_v4. Both systems utilized different neural retrieval approaches: Triad_v3 used dense embeddings (`all-MiniLM-L12-v2`), while Triad_v4 employed `cross-encoder/ms-marco-electra-base` (as discussed in section 4.2).
3. **Presentation of the top 10 results for Queries 1 and 3** for both Triad_v3 (the best performer) and Triad_v4.

1. Overall Performance Compared to Assignment 1

The models implemented in this assignment significantly outperformed those in Assignment 1. The primary reason for this improvement is the introduction of a hybrid approach in the **triad_v3** model, which combines multiple retrieval and ranking techniques. While the models in Assignment 1 primarily relied on BM25 retrieval and basic re-ranking, **triad_v3** leverages query expansion, embedding-based retrieval, and cross-encoder ranking, which leads to more accurate document retrieval and relevance scoring.

Best Performing Models

Among the models tested, **triad_v3** was the top performer, with models using query expansion with BM25 also showing strong results. The combination of BM25 and embedding-based retrieval, followed by cross-encoder re-ranking, led to more relevant and precise document retrieval.

Comparison with previous IR system discussed in Assignment 1

The triad_v3 model outperformed the models from Assignment 1 due to the following factors:

Hybrid Retrieval and Ranking:

- **BM25 Retrieval**: Initial retrieval is performed using BM25, which effectively ranks documents based on term frequency and inverse document frequency.
- **Embedding-Based Retrieval**: Cosine similarity between the embedded query and document representations further refines the initial BM25 results by capturing semantic relationships missed by traditional lexical matching.
- **Cross-Encoder Re-Ranking**: The top-ranked documents are re-ranked using a cross-encoder, which computes relevance at a finer granularity by considering the query-document relationship in detail.

Query Expansion Using LLM:

- The model uses Google's Gemini LLM to generate multiple query variations, enabling the retriever to capture a broader range of relevant documents.
- This approach reduces the impact of query ambiguity, resulting in improved recall.

Weighted Scoring Between BM25 and Embedding Retrieval:

- The model assigns a higher weight to embedding scores (0.7) compared to BM25 scores (0.3), allowing semantic similarity to play a dominant role in document selection.
- This weighting mechanism balances the strengths of both approaches, enhancing overall retrieval performance.

Sentence-Level Document Splitting and Deduplication:

- Documents are split at the sentence level with overlap, allowing finer granularity during retrieval and ranking.
- The deduplication function ensures that only the most relevant chunk from each document is retained, avoiding redundant information.

2. Neural Models Used for Improved Retrieval

In this section, we discuss the two advanced neural retrieval models that were implemented: **Triad_v3** and **Triad_v4**, which extended earlier hybrid models by incorporating neural rankers and improving document

processing.

Triad_v3

- **Overview:**
  Triad_v3 builds upon Triad_v2 by introducing document chunking to mitigate issues where long documents were truncated by the embedding and ranking models. Documents were split into chunks of **3 sentences with 2-sentence overlap**, ensuring that relevant content was captured without losing context.
  After retrieval and ranking, duplicate documents were removed to improve result diversity.

- **Components:**
  - **BM25:** Initial term-based retrieval.
  - **Embeddings:** Sentence embeddings using `sentence-transformers/all-MiniLM-L12-v2` to enhance semantic relevance.
  - **Semantic Ranker:** Cross-encoder model `sentence-transformers/all-MiniLM-L12-v2` used to refine rankings.

- **Performance:**
  - **MAP:** 0.6605
  - **P@10:** 0.0943

Triad_v3 achieved the highest MAP, indicating strong relevance in the ranked documents. The document chunking approach prevented the truncation of longer documents and contributed to improved performance.

Triad_v4

- **Overview:**
  Triad_v4 is an extension of Triad_v3 that introduces a **larger and more sophisticated ranking model** (`cross-encoder/ms-marco-electra-base`). This model improves the ranking of candidate documents by providing more accurate semantic relevance judgments. Additionally, the document splitting strategy was adjusted to **2 sentences with 1-sentence overlap** to reduce processing overhead and capture finer-grained content.

- **Components:**
  - **BM25:** Initial term-based retrieval.
  - **Embeddings:** Same `sentence-transformers/all-MiniLM-L12-v2` embedding model as Triad_v3.
  - **Semantic Ranker:** Larger cross-encoder model `cross-encoder/ms-marco-electra-base` for improved relevance evaluation.

- **Performance:**
  - **MAP:** 0.6583
  - **P@10:** 0.0947

Triad_v4 demonstrated slightly higher P@10, suggesting improved precision in the top 10 results. However, its MAP was marginally lower than Triad_v3, likely due to the finer-grained document splitting that may have reduced overall context.

Comparison and Insights

- Both models significantly outperformed earlier systems, particularly in terms of MAP and P@10.
- **Triad_v3** excelled in achieving the highest MAP due to its balanced document splitting and effective deduplication.
- **Triad_v4** achieved better precision in the top 10 results by using a more complex ranker and finer document splitting, although the trade-off was a slight dip in overall MAP.

These models illustrate the impact of incorporating neural rankers and optimized document processing techniques on retrieval performance.

3. First 10 results for queries 1 and 3

Triad_v3 system (`all-MiniLM-L12-v2`)

The following tables provide the top 10 results for queries 1 and 3 using Triad_v3 IR system

Query1

Query ID	Q0	Document ID	Rank	Similarity Score	Run
1	Q0	40212412	1	0.001539	run1
1	Q0	43385013	2	0.001450	run1
1	Q0	6863070	3	0.000552	run1
1	Q0	10931595	4	0.000499	run1
1	Q0	13231899	5	0.000243	run1
1	Q0	27049238	6	0.000241	run1
1	Q0	1065627	7	0.000222	run1
1	Q0	16736872	8	0.000149	run1
1	Q0	31543713	9	0.000080	run1
1	Q0	1346695	10	0.000073	run1

Query 3

Query ID	Q0	Document ID	Rank	Similarity Score	Run
3	Q0	14717500	1	0.983919	run1
3	Q0	2739854	2	0.896134	run1
3	Q0	4414547	3	0.851782	run1

Query ID	Q0	Document ID	Rank	Similarity Score	Run
3	Q0	4378885	4	0.543078	run1
3	Q0	13914198	5	0.540342	run1
3	Q0	19058822	6	0.457236	run1
3	Q0	23389795	7	0.456205	run1
3	Q0	1388704	8	0.425642	run1
3	Q0	4632921	9	0.269687	run1
3	Q0	2107238	10	0.182091	run1

Actual Document for Query 1: 31715818

Observation: The correct document was not retrieved within the top 10 results. The highest-ranking document had a similarity score of only 0.001539, and none of the retrieved documents match the correct document. After further investigation, we figured that the correct document was retrieved at rank 74 with a similarity score of 0.000017.

Actual Document for Query 3: 14717500

Observation: The correct document was retrieved at rank 1 with a high similarity score of 0.983919, indicating a successful retrieval.

Triad_v4 (`cross-encoder/ms-marco-electra-base`)

The following tables provide the top 10 results for queries 1 and 3 using the Triad_v4 IR system.

Query 1

Query ID	Q0	Document ID	Rank	Similarity Score	Run
1	Q0	40212412	1	0.002324	run1
1	Q0	10931595	2	0.001993	run1
1	Q0	2060137	3	0.000420	run1
1	Q0	4346436	4	0.000285	run1
1	Q0	7583104	5	0.000158	run1
1	Q0	1065627	6	0.000132	run1
1	Q0	5956016	7	0.000121	run1
1	Q0	14103509	8	0.000079	run1
1	Q0	6128334	9	0.000077	run1
1	Q0	41329220	10	0.000065	run1

Query 3

Query ID	Q0	Document ID	Rank	Similarity Score	Run
3	Q0	14717500	1	0.905012	run1
3	Q0	4378885	2	0.194825	run1
3	Q0	19058822	3	0.092834	run1
3	Q0	2739854	4	0.078753	run1
3	Q0	23389795	5	0.023050	run1
3	Q0	3222187	6	0.018730	run1
3	Q0	12271486	7	0.017602	run1
3	Q0	4414547	8	0.012884	run1
3	Q0	14729253	9	0.011318	run1
3	Q0	20375264	10	0.010078	run1

Observations:

- **Query 1:**
    - The actual document (ID: 31715818) did not appear within the top 10 results for Triad_v4. It was retrieved at rank 46.
- **Query 3:**
    - The actual document (ID: 14717500) was retrieved at rank 1 with a similarity score of 0.905012, indicating a successful retrieval.
    - The remaining documents are less relevant with a significant drop in similarity scores, but the top result was highly relevant.

These results show that Triad_v4 achieved a successful retrieval for Query 3, but for Query 1, the correct document was not retrieved within the top 10, highlighting some limitations in the retrieval performance for certain queries.

Comparison for Query 1:

- **Rank Comparison**: Both Triad_v3 and Triad_v4 retrieve the same document (ID: 40212412) at rank 1, but the similarity score in Triad_v4 (0.002324) is higher than in Triad_v3 (0.001539). However, neither system retrieves the correct document (ID: 31715818) within the top 10.
- **Relevance**: Both systems return similar results, with some overlap in document IDs, but none of the results in the top 10 for either system are highly relevant based on the actual document for Query 1.

Comparison for Query 3:

- **Rank Comparison**: Both Triad_v3 and Triad_v4 correctly retrieve the actual document (ID: 14717500) at rank 1, but the similarity score for Triad_v3 (0.983919) is higher than for Triad_v4 (0.905012).
- **Relevance**: The first-ranked document in both systems is highly relevant, and both systems seem to perform well in terms of retrieving the correct document. However, the relevance of the remaining

documents in Triad_v4 is much lower compared to Triad_v3.

Summary of the Comparison:

- **Triad_v3** generally produces higher similarity scores for the top-ranked documents, especially for **Query 3**, where the correct document is retrieved at rank 1 with a very high similarity score (0.983919).
- **Triad_v4** performs similarly to **Triad_v3** for **Query 3**, but for **Query 1**, neither system retrieves the correct document within the top 10, and the relevance of the results is relatively low.
- Both systems demonstrate strengths in **Query 3** by retrieving the correct document, but **Triad_v3** seems to perform better overall in terms of similarity scores and relevance ranking.

This comparison highlights that while both systems have their strengths, Triad_v3 performs slightly better in terms of overall retrieval quality, especially in terms of similarity scores for relevant documents.

Conclusion

In this report, we compared the performance of various IR systems, focusing on Triad_v3 and Triad_v4. Both systems used different neural retrieval models: Triad_v3 employed `all-MiniLM-L12-v2` dense embeddings, while Triad_v4 utilized the `cross-encoder/ms-marco-electra-base` ranking model.

We also compared the performance of Triad_v3 with the original Assignment 1 IR system. The results showed significant improvements, with the Mean Average Precision (MAP) increasing from 0.5485 (Assignment 1) to 0.658 (Triad_v3), a 20% improvement. Similarly, Precision at Rank 10 (P@10) increased from 0.0847 (Assignment 1) to 0.0943 (Triad_v3), marking an 11% improvement.